

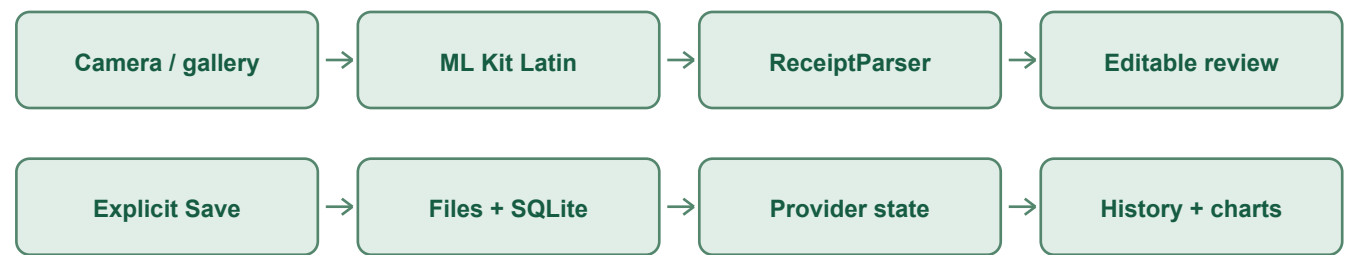
ReceiptWise

Technical report | Android | Version 1.1.0+3 | 3 October 2026

1. Product and scope

ReceiptWise turns paper receipts into local expenses. Users capture with the live scanner or the system camera, choose an existing receipt image, or enter an expense manually. ML Kit recognizes Latin text on the device. Every extracted result is reviewed and validated before an explicit Save. No cloud OCR, account or paid API is used.

2. Architecture and data flow



The existing UI / services / repository / database / models separation remains. A small Provider layer exposes one immutable SQLite snapshot to Home, History and Analytics. Camera and OCR own their native resources; services own parsing, image processing and write coordination. Navigator retains the established routes.

Layer	Responsibility
UI and app	Material 3, editable forms, navigation, light/dark/system appearance and user confirmation.
Services	Camera, picker recovery, native-resolution image processing, OCR, regex parsing, safe save/delete and analytics aggregation.
State	TransactionState (Provider + ChangeNotifier): records, loading/error, summary and chart data; AppearanceState: current theme.
Repository / database	sqlite CRUD and range queries, schema version 1, lazy connection with coordinated close/open.

3. Implementation stack

Flutter 3.47.5 / Dart 3.13.4; minimum project Flutter 3.44 / Dart 3.12; Android API 24+. Packages: camera, image_picker, google_mlkit_text_recognition, sqlite, provider, image, path_provider and path. Charts use Flutter Canvas and CustomPainter. Host persistence tests use SQLite FFI; an Android integration scenario uses native sqlite.

OCR, parsing and storage

4. OCR and image handling

The live scanner snapshots its frame geometry, captures once and maps preview cover/orientation into original image pixels. Cropping and JPEG encoding run outside the UI in an isolate. Picker images retain the full image because no scanner frame applies; EXIF orientation is baked into a temporary JPEG. Android lost-picker data is recovered for review, never automatically saved.

OcrService owns a lazy TextRecognizer using TextRecognitionScript.latin and InputImage.fromFilePath. Requests are serialized and disposal waits for accepted work. Full text, blocks and lines are returned. OCR failure still opens manual review. Stopwatch duration is logged only in debug; no fixed latency guarantee is made.

5. ReceiptParser rules

Field	Heuristic and safe fallback
Merchant	First plausible header after excluding invoice/contact/address/date/total lines. Original accents are preserved.
Amount	Accent-folded final-total labels outrank item totals. Accept a standalone next-line amount. Exclude subtotal, cash tendered and change from currency fallback. Support 150.000, 150,000, 150 000 and VND / VNĐ / đ.
Date	Strict day-first and year-first formats with consistent slash, hyphen or dot separators. Prefer labelled dates; reject impossible dates.
Review	Nullable extraction remains blank. Require merchant, positive finite amount, valid date and one of six categories. Nothing saves before confirmation.

6. SQLite and receipt ownership

transactions: id INTEGER PRIMARY KEY AUTOINCREMENT; merchant TEXT NOT NULL; amount REAL NOT NULL; date TEXT NOT NULL; category TEXT NOT NULL; receipt_image_path TEXT; created_at TEXT NOT NULL. A date index supports range queries. DateTime values use UTC ISO-8601 strings and local display.

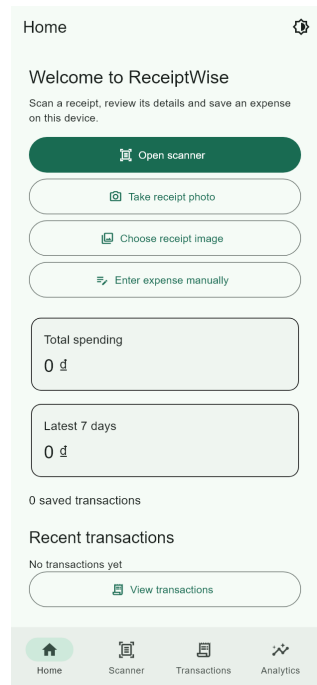
Save copies the temporary JPEG to a unique documents path, then inserts its path into SQLite. Insert failure rolls back only the uncommitted copy and retains the source. Manual records have a null image path. Edits retain image/creation metadata. Confirmed deletion removes the row first, then the file; file cleanup can be retried without repeating the SQL deletion.

7. CustomPainter analytics

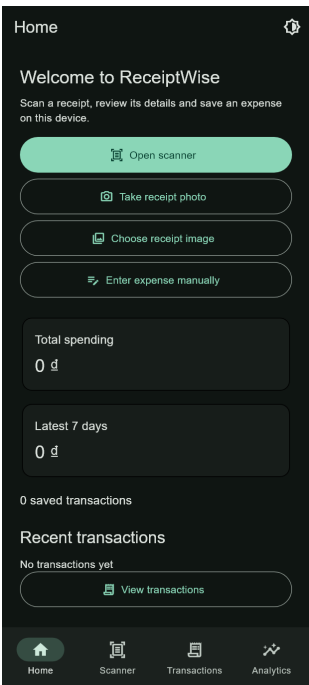
CategoryDonutPainter draws animated arcs for Food, Study, Travel, Gear, Entertainment and Other. WeeklyBarPainter draws animated rounded bars for the latest seven local calendar days. Aggregation occurs in AnalyticsService before painting. BigInt cents avoid total overflow; painters receive finite normalized geometry. Repaint checks compare drawing data, colors and progress. Reduced motion and zero/empty states are supported.

UI evidence

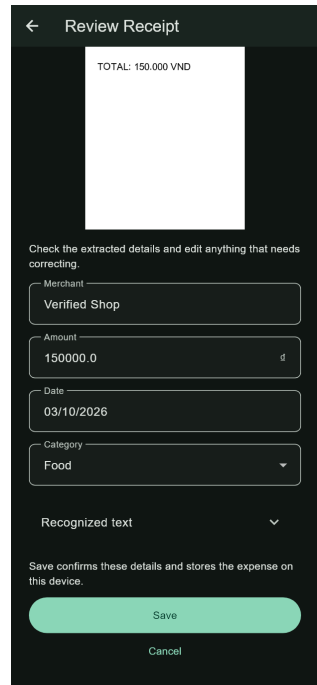
Actual Flutter-rendered screens from the automated host flow. Fixture receipt and OCR text are synthetic test inputs; these are not physical-device screenshots or native OCR accuracy evidence.



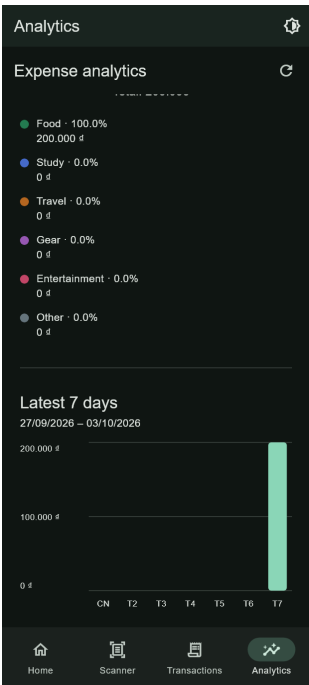
Home - light appearance



Home - dark appearance



Editable review before Save



Latest-seven-day CustomPainter bars

Verification and release

8. Automated evidence

Run `dart format .`, `flutter pub get`, `flutter analyze` and `flutter test`. The shared receipt-flow scenario covers explicit review, a real SQLite insert, Provider refresh, editing, category/daily analytics, deletion and image-cleanup retry. Import tests cover camera/gallery selection, cancellation, invalid data rollback, OCR failure fallback and Android lost-data handling with test doubles.

The same scenario is available under `integration_test/receipt_flow_test.dart` for native Android `sqlite`. Camera/image selection and OCR boundaries are replaced by deterministic fixtures in this scenario. Separate physical tests must establish actual camera and ML Kit behavior. Latest command results and the APK checksum are recorded in `README.md` and `COURSE_READINESS.md`.

9. Release signing and installation

The release uses application ID `com.anhvy.receiptwise` and a dedicated RSA signing key, with credentials in ignored local files. Release builds require private signing configuration and do not silently use the debug certificate. Secure backups are required for future updates. The application ID differs from older sample debug builds; it installs separately and does not migrate their data.

10. Privacy and limitations

OCR text is used in memory for review and is not persisted. Images and transactions remain local; application code uploads neither. SQLite/images are not separately encrypted; Android/device backup policy still applies. Recognition depends on lighting, focus and receipt layout. The parser is heuristic and cannot prove correctness. Screenshot/payment parsing, bilingual UI and iOS support are outside this course-core upgrade.

11. Remaining acceptance checks

Deliverable	Current evidence / remaining action
Core source and tests	Implemented, automated checks documented. GitHub source is maintained in the user's repository.
Signed release APK	Dedicated signing configured; verify signature after build. Actual installation still needs an Android device.
Device demonstration	No Android device was connected during this work. Test permission, camera/torch/focus, rotation, gallery recovery, airplane-mode OCR and restart persistence.
Screenshots / video	Host-rendered fixture screenshots included. Replace/add real device captures; record a 2-3 minute demo using <code>docs/DEMO_CHECKLIST.md</code> .

References and project documents

ARCHITECTURE.md; COURSE_READINESS.md; docs/DEMO_CHECKLIST.md; docs/RELEASE_SIGNING.md. Scope reference: Vcoch27/receipt-ocr-expense-tracker course PRD at commit `c473bc5`. Project source: github.com/anhvy0904/receipt-ocr-expense-tracker.